

# WALKIE TALKIE MESH CHAT

*by John Dovey*

# TABLE OF CONTENTS

|                             |    |
|-----------------------------|----|
| Title Page                  | i  |
| Chapter 1: Getting Started  | 1  |
| Chapter 2: The Base Station | 2  |
| Chapter 3: Windows Builds   | 7  |
| Chapter 4: macOS builds     | 8  |
| Chapter 5: Android Client   | 9  |
| Chapter 6: The iPhone app   | 12 |
| Notes                       | 13 |

# GETTING STARTED

WalkieTalkie is a cross-platform push-to-talk (PTT) app. This chapter covers the desktop "Base Station" — the Go server app for Mac, Windows, and Linux — and the Android and iPhone apps. Android + Base Station are confirmed working together on real hardware. The iPhone app is in progress (buildable scaffold; device mesh verify pending).

# CHAPTER 2: THE BASE STATION

## DESKTOP

### WHAT A BASE STATION IS

Every desktop instance of WalkieTalkie is called a **Base Station**. A Base Station is not just a monitoring dashboard — it's a full participant: it can talk and listen like any other device, and it also hosts a web dashboard showing every device it has seen on the network.

Each Base Station names itself automatically, as "**Base Station: <your computer's name>**" — for example, "Base Station: Johns-MacBook-Air". There's currently no way to rename it from the dashboard; this is intentional (see Settings below).

### RUNNING A BASE STATION

From the server/ directory of the project:

```
go run .
```

This starts the Base Station listening on port **9091** by default. Open <http://localhost:9091> in a browser to see the dashboard. On desktop builds a small **WT** system-tray menu offers **Open Dashboard** and **Quit**; use `-no-tray` for headless / SSH / CI runs.

**There is no login.** Anyone who can reach the port — anyone on the same LAN, by default — can view the dashboard, including every device's live GPS location if it has one. Don't expose this port beyond a network you trust.

### TALKING FROM A BASE STATION

The big green **Hold to Talk** button at the top of the dashboard's home page lets a person sitting at (or on the same LAN as) the Base Station machine actually use its mic and speaker — press and hold to transmit, release to stop, exactly like the on-screen button in the Android app.

### THE DEVICE LIST

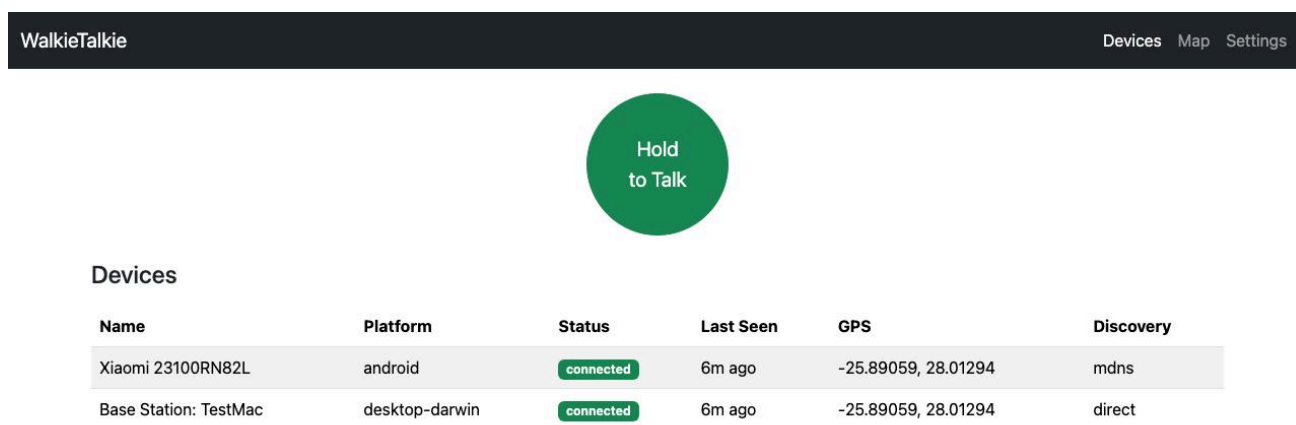
The dashboard's home page (/) lists every device the Base Station has seen recently: its name, platform,

connection status (Connected/Disconnected), how long ago it was last seen, its GPS location if known, and how it was discovered (directly, over the LAN via mDNS, or reported by another device over Bluetooth). The list refreshes automatically every few seconds, sorted most-recently-seen first, and each device's name is colored green if it's currently connected or gray if it's disconnected.

Next to each device are two small action icons: **Voice Message** (always available) opens a recorder so you can send an async voice note that the Base Station holds until that device connects again (notes are kept for at least 21 days, then deleted). **Invite to Chat** appears only for connected devices and starts a private 1:1 channel. If a device has voice notes waiting for it, the list shows how many.

The **Chats** button on the left of the nav opens a drawer with active private channels (and unread counts) and a list of senders who have new voice messages for you. Opening a voice-message entry shows the thread — play, reply, or delete. Opening a private channel switches the Talk control to that pair only; while you are not focused on the channel, clips from the other party queue as voice notes until you return.

When two Base Stations are running on the same LAN, they discover each other the same way any two devices do — automatically, with no setup — and each will list the other, including itself as a normal entry in its own list. They also keep their device lists in sync with each other continuously in the background (see "Multiple Base Stations" below).



Base Station Dashboard

This is a real screenshot from an actual test session: a Base Station running on a Mac, talking to a real Android phone over Wi-Fi.

## OLD NODES

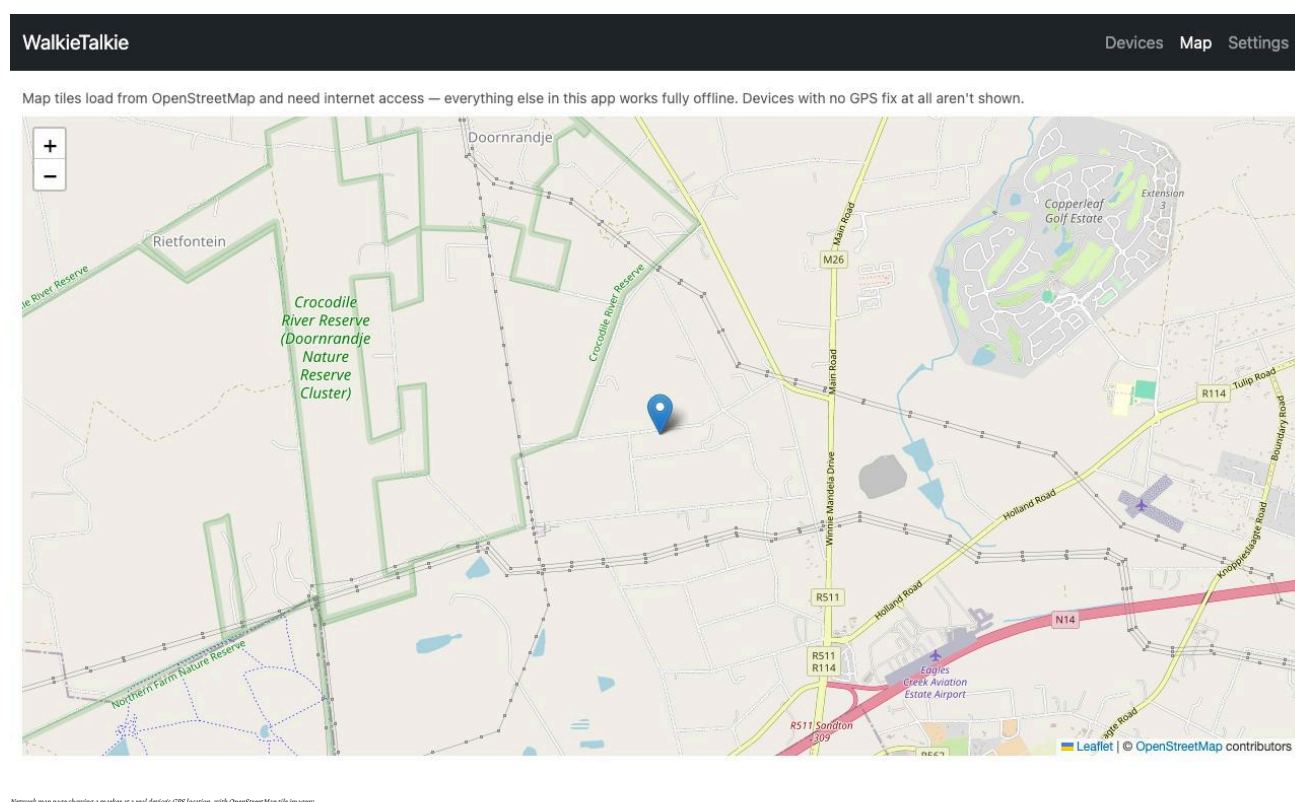
Any device not seen in over 2 hours moves off the main device list onto its own **Old Nodes** page (/old-nodes) instead of cluttering up the main page or disappearing entirely — this is where a device that crashed, went

out of range, or was leftover test data ends up, still sorted most-recently-seen first. The moment that device is seen again, it moves straight back to the main page on its own; nothing needs to be done manually.

## THE MAP

The **Map** page (/map) plots every device with a known GPS position. It needs internet access to load the map imagery itself (from OpenStreetMap) — this is the one page in the app that isn't fully offline; everything else, including the device list and Talk button, works with no internet connection at all.

A Base Station machine usually has no GPS hardware of its own. Instead, its position on the map is *estimated* as the average location of every device currently connected to it that has a GPS fix — so it'll only show up once at least one connected device is reporting its location.



## MULTIPLE BASE STATIONS

If you run more than one Base Station on the same LAN, they don't just discover each other — they continuously synchronize their device lists with each other in the background, so a device seen by only one Base Station still shows up correctly on the others within a few seconds. If the two Base Stations ever disagree about a device (e.g. its last-seen time or GPS), whichever one saw it most recently wins.

## SETTINGS

The **Settings** page (/settings) lets you change the Base Station's own configuration: the port it listens on, how often GPS updates are expected, how often it re-syncs with other Base Stations, the relay threshold, and

whether the relay fallback is enabled. Changing the port restarts the web server on the new port — the page tells you the new address to reconnect to.

The **relay threshold** (10 by default) controls when audio switches from direct device-to-device WebRTC to routing through this Base Station's SFU instead, so a large mesh does not stay fully peer-to-peer. When **Relay enabled** is on and the number of connected peers is already at or above the threshold, *new* connections join via the SFU (also used if a direct ICE attempt fails). Turn Relay enabled off to stay P2P-only. Live private-channel SFU is a separate, later feature; group mesh relay is what this setting drives today.

The **"Mark disconnected after"** setting (180 seconds by default) controls how long a device can go unseen before it's automatically marked Disconnected, even if it never sent a proper disconnect notice — this is what clears out a device that crashed, walked out of range, or was leftover test data, instead of leaving it stuck showing "connected" forever.

Fixed a related bug where a device's GPS position, name, and app version could get stuck at whatever they were the first time it was discovered, never updating again for the rest of the Base Station's run — even though the device kept sending fresh updates. Both this and the "connected forever" issue above were tracked down after noticing a leftover test entry that wouldn't go away and a phone's map position that never moved.

Settings does **not** let you rename a device from the browser. Every device — including a Base Station — sets its own name locally; the dashboard only ever displays what a device has told it about itself.

WalkieTalkie

DevicesMapSettings

### Settings

Port

9091

Changing this restarts the web server on the new port.

GPS update interval (seconds)

30

Base Station sync interval (seconds)

30

How often this Base Station re-syncs its device list with other Base Stations it finds on the LAN.

Relay threshold (connected peers)

10

Once this many peers are connected at once, route all audio through the relay instead of direct connections.

☒ Enable relay fallback for unreachable peers

Save

*Settings page showing port, GPS interval, sync interval, relay threshold, and relay toggle fields*

The **Stats** page (/stats) shows usage and traffic for this Base Station: nodes newly seen, private-channel invites/accepts, voice notes and channel clips (counts and bytes), and live group-PTT bytes sent/received by this machine. Ranges: today, week, month, or all time. Peer-to-peer audio that never passes through this Base Station is not included.

## ABOUT

The **About** page (/about) shows this Base Station's own name, platform, version, and device ID. The device list on the home page also shows the app version each connected device announced itself with, so a mismatched-version device is easy to spot at a glance.

## VOICE MESSAGES AND PRIVATE CHANNELS

Both the Base Station dashboard and the Android app support **async voice messages** and **invite-only private channels**. Clips are stored on a Base Station on the LAN (not sent peer-to-peer yet — that is a planned follow-up).

- On each device row, a microphone icon sends a **Voice Message** to that node (works even if they are offline — the note is held until they return, for at least 21 days, then deleted). A chat icon **Invite to Chat** appears only while the peer is connected.
- A left-hand drawer lists active private channels and nodes with new voice messages waiting, with count badges.
- Opening a voice-message badge shows the thread for that sender: play, reply, or delete notes.
- Recording uses Start / Stop & Send / Cancel. Private-channel talk uses the same Hold-to-talk gesture as the main screen, but clips are scoped to that pair and queue with a badge when the recipient is not focused on the channel.
- The device list shows how many voice notes are waiting for each node.

Voice notes require a reachable Base Station. If none is discovered on the LAN, send fails with a clear error rather than dropping silently.

## RUNNING MORE THAN ONE INSTANCE (FOR TESTING)

The desktop app accepts a few flags useful when testing with more than one instance on a single machine:

- `--port`
- `--data-dir`
- `--name`
- `--no-audio` (skip mic/speaker setup, so two instances on one machine don't fight over the same audio hardware)

# CHAPTER 3: WINDOWS BUILDS

## WINDOWS BUILDS

A Windows Base Station is built by cross-compiling from the Mac dev machine — there's no need for an actual Windows machine to produce the .exe. Run `./tools/build-windows-server.sh` from the repo root; the first run takes a few extra minutes to set up a one-time cross-compile toolchain, after which it just rebuilds the executable. The resulting .exe has full mic/speaker support, not just the registry/web-UI/relay pieces. This hasn't been tested on real Windows hardware yet — only confirmed to be a valid Windows executable with sane-looking dependencies.

# CHAPTER 4: MACOS BUILDS

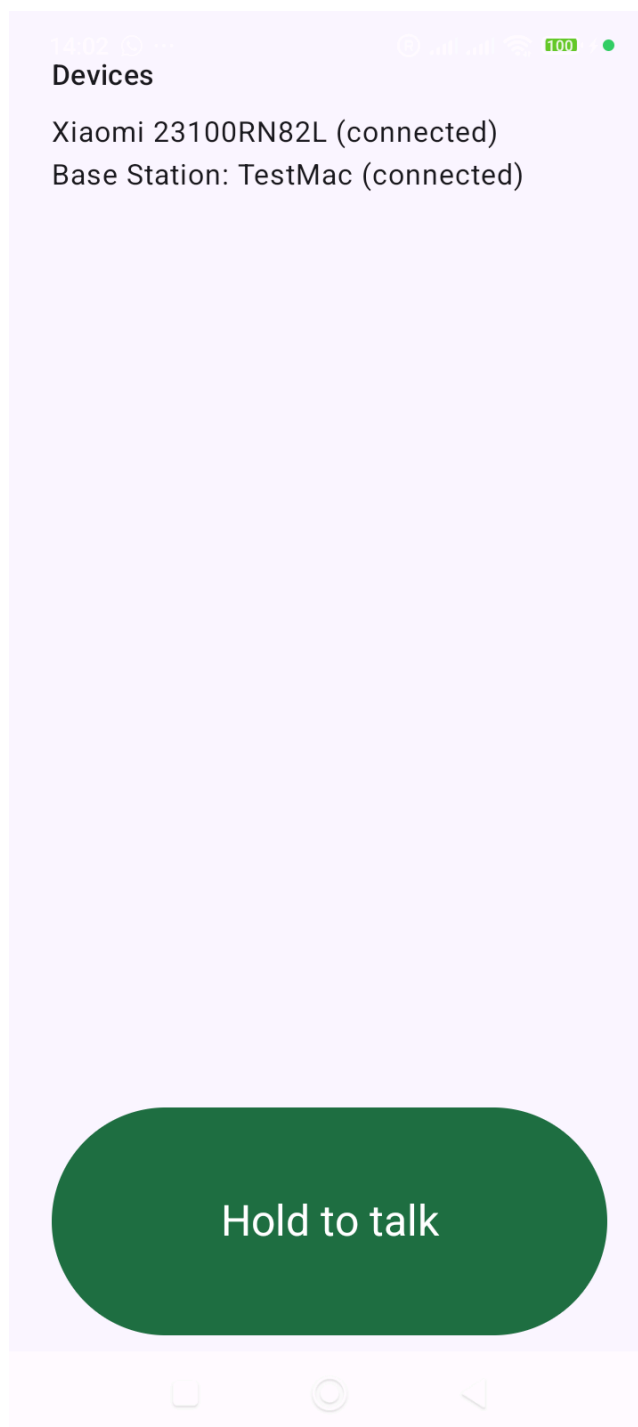
## MACOS BUILDS

A macOS Base Station binary for **both** Apple Silicon and Intel is produced by `./tools/build-macos-server.sh` from the repo root. The first run builds a per-architecture Opus toolchain (cached afterward). It writes three files under `/Volumes/JohnDovey/tmp` by default: `walkietalkie-server-darwin-arm64`, `walkietalkie-server-darwin-amd64`, and a universal `walkietalkie-server-darwin-universal` that runs on either chip. All include full mic/speaker support.

# CHAPTER 5: ANDROID CLIENT

## THE ANDROID APP

The Android app was confirmed working on real hardware for the first time on 2026-07-13: it discovers a Base Station over Wi-Fi, connects to it, and exchanges real two-way audio — confirmed in both directions, including someone pressing the phone's Talk button and hearing their own voice come out of the desktop machine's speakers.



A few things worth knowing if you're trying this yourself:

- The app needs microphone, location, and Bluetooth permissions — grant all of them when asked, or nothing will work.
- On some phones (confirmed on a Xiaomi/HyperOS device), aggressive battery management kills the app's network activity a few seconds after connecting, even though it's a proper foreground service. If audio drops after several seconds, check your phone's battery settings and allow the app to run unrestricted / disable battery optimization for it.

The app has a left **Chats** drawer plus screens switched via the top links: **Devices** (the list of other devices, per-device **VM** / **Chat** actions, and the Hold to talk button), **Settings** (set a Nickname/User-name for this device —

leave it blank to fall back to the phone's manufacturer and model), and **About** (this app's own version, device ID, and discovered Base Station URL).

Voice messages and private channels need a Base Station on the LAN; without one, group Hold-to-talk still works, but VM/Chat will fail until a Base Station appears.

The Devices list is sorted most-recently-seen first, and — unlike the desktop Base Station, which keeps every device it's ever seen — the phone automatically forgets (deletes) any device it hasn't seen in over 48 hours, to keep the on-device list from accumulating clutter over time.

Not yet tested: whether a third device can detect the phone over Bluetooth and report its presence to a Base Station when the phone has no Wi-Fi — that needs a second Bluetooth-capable device to verify.

# CHAPTER 6: THE IPHONE APP

## THE IPHONE APP

The iPhone app (Phase 4) is a SwiftUI shell around the same Go core as Android. Version starts at **0.1.0**. Before opening the Xcode project, run `tools/gomobile-bind-ios.sh` and `tools/build-opus-ios.sh` (the XCFrameworks are gitignored, like the Android AAR).

- Grant **microphone**, **Bluetooth**, and **location** when asked — discovery, GPS, and Talk need them.
- **Hold to Talk** on the Devices screen works in the foreground (and is the fallback on Simulator / builds without a paid Push to Talk team).
- Locked-screen PTT uses Apple's **PushToTalk** framework (PTChannelManager). That entitlement needs a paid Apple Developer team; set `DEVELOPMENT_TEAM` in a gitignored `ios/Config/Local.xcconfig` (see the example next to it).
- Battery: iOS is generally less aggressive than HyperOS about killing foreground audio, but Background App Refresh / Low Power Mode can still pause mesh activity if the app is fully backgrounded without an active PTT session — keep the app open while testing LAN Talk.

Voice messages and private channels use the same Base Station APIs as Android; without a Base Station on the LAN, group Hold-to-talk still works, but the Chats drawer remains limited.

# NOTES